In-Database Feature Store for Lightweight Analytical Databases
CP2107 Final Report

Isaac Ong

National University of Singapore

Advised by Zeng Lingze

***Abstract*—Conventional feature stores split computation across change-data-capture, stream processing, an online key-value store and an offline warehouse — an architecture that presumes cloud infrastructure and rules out edge deployment. We ask whether a single RDBMS can instead act as a self-contained feature store, and present a proof-of-concept inside DuckDB exposing `CREATE`, `REFRESH` and `SERVE FEATURE` as first-class SQL statements. We analyze the cost of the joins associated with feature stores, describe the design architecture of our implementation, and present four result-preserving optimizations. Over 55 scenarios on a ClickBench-derived clickstream of up to 100M rows, these optimizations cut suite runtime from 211.3 s to 34.9 s, an 83.5% reduction.**

1. Introduction

A feature store is the system that computes, stores and serves *temporal features* — such as quantities aggregated over a rolling window of an entity's history — for two downstream consumers: training a model, and serving the feature at inference time. For example, a bank performing credit-card fraud detection might train a model on each cardholder's Max/Min/Avg/Top2 transactions in the last 10 seconds; then, when scoring a live application, the feature store looks up that same feature computed as of the request time to get a prediction (Figure 1).

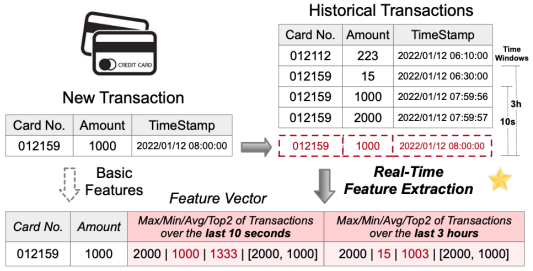


Figure 1. Example of feature stores being used for credit card analytics. Adapted from the FEBench benchmark [1].

In a conventional enterprise ML stack, the pipeline that computes these features is separate from the database holding the raw data: an *offline store* is populated by batch jobs that compute windowed aggregates for training, while a separate *online store* is populated to serve low-latency feature lookups during inference, typically via a Change-Data-Capture → Flink → Redis pipeline (Figure 2).

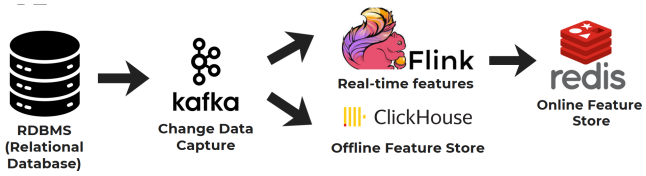


Figure 2. The conventional enterprise feature pipeline, built for large-scale cloud deployments. Architecture adapted from the CDC—Flink pipeline described in FEBench [1] and the offline/online store separation from Hopsworks [2].

This split architecture gives rise to several issues:

1) **Training/serving skew:** the feature is implemented twice, often in different languages or systems, and the two definitions may drift over time.
2) **Edge deployment:** IoT gateways and embedded systems often cannot host a distributed cloud stack of CDC, Kafka, Flink, Redis, and an OLAP warehouse.
3) **Resource constraints:** small teams with limited budgets cannot operate and maintain a multi-component distributed pipeline.
4) **Network round-trips:** low-latency local inference needs features co-located with the data and the model on the same node.

These challenges motivate a proof-of-concept: can a single RDBMS serve as a self-contained feature store, computing, storing and serving features without an external infrastructure? To investigate this, we forked DuckDB to add native

feature DDL and DML[1], housing the offline and online stores in one analytical database.

DuckDB was chosen because four of its existing properties map directly onto what a feature store needs.

1) **Native `ASOF JOIN`:** Its native `ASOF JOIN` The correctness of online serving can be delegated to an operator that is already implemented and tested.
2) **Vectorized execution:** Reduces windowed aggregate of a refresh feature operation to a parallel group-by.
3) **Per-row-group zonemaps:** Lets range predicates skip whole row groups.
4) **Multi-Version Concurrency Control:** allows catalog metadata, snapshot appends and garbage collection to commit as a single transaction.

2. Feature Operations

The full SQL surface is summarized in Listing 1. Of these statements, two do the actual work of a feature store: `REFRESH FEATURE`, which materializes the offline snapshot, and `SERVE FEATURE`, which answers the online lookup; both reduce to a single join. The remaining statements are catalog and retrieval plumbing, and are covered in §3.

```
CREATE FEATURE [IF NOT EXISTS] <name>
   ENTITY <entity_table> [(<key_col>, ...)]
   TIMESTAMP <ts_col>
   [WINDOW <interval>] [TTL <interval>]
   [EVERY <interval>] [RETAIN <n>]
   AS (<select_query>);
ALTER FEATURE <name> SET <clause> <value>;
REFRESH FEATURE <name> [AT '<timestamp>'];
SELECT * FROM FEATURE <name> AT VERSION 3;
SERVE FEATURE <a> FOR <spine>; -- online
SERVE FEATURES <a>, <b> FOR <spine> ASOF <ts>;
```

Listing 1. The feature store SQL surface.

2.1. `REFRESH`: a Point-in-Time Join

2.1.1. The Query. A refresh computes, for *every* entity, the windowed aggregate over events in the half-open interval $[ts - w,\ ts)$ and appends the result tagged with a version and timestamp; the exclusive upper bound is what makes this a snapshot of what was knowable at ts. The rewrite (Listing 2) joins `LEFT` from the entity dimension, and additive aggregates (such as SUM and COUNT) are coalesced to zero and others are left as NULL.

```
SELECT e.<key>, COALESCE(agg.__f0, 0) AS <feature>
FROM (SELECT [DISTINCT] <keys> FROM <entity>) e
LEFT JOIN ( <the user's AS (...) query>
   WHERE <ts_col> >= TIMESTAMP '<t>' - INTERVAL <w>
     AND <ts_col> < TIMESTAMP '<t>' ) agg
  ON e.<key> = agg.<entity_col>;
```

Listing 2. The generated snapshot query, abridged.

1. https://github.com/IZO-Ong/duckdb-fs

2.1.2. Cost. Let N be the number of rows in the event table, $N_w \leq N$ the number of events falling inside the window $[ts - w,\ ts)$, and E the number of entities in the entity dimension. Listing 2 decomposes into three stages:

1) *Windowed scan and aggregate.* The window predicate is a range filter on the timestamp column, so the scan is $O(N)$; because the event table is clustered by timestamp, zonemaps (§4.2) let DuckDB skip non-overlapping row groups and bring this down towards $O(N_w)$. Grouping the surviving rows by entity is a hash aggregate, $O(N_w)$ expected, emitting at most $\min(N_w, E)$ groups.
2) *Entity column.* Projecting the entity keys costs $O(E)$; the `DISTINCT` is elided when the projection covers a `PRIMARY KEY` (§4), otherwise it is a hash aggregate, also $O(E)$ expected.
3) *Left join.* A hash `LEFT JOIN` builds over the $\min(N_w, E)$ aggregate groups and probes with the E rows, so $O(N_w + E)$ expected.

The total is therefore

$$O(N_w + E) \ = \ O(\max(N_w,\ E))$$

2.2. `SERVE`: an ASOF Join

2.2.1. The Query. Serving matches each row of a *spine* — the incoming transaction table of entities and request timestamps for which features are being requested — to that entity's most recent snapshot at or before the request time. Engines in other databases such as SQLite lack a native ASOF operator and thus must emulate it, either with a correlated subquery or with an inequality join. However, because DuckDB exposes ASOF joins natively, a single `ASOF LEFT JOIN` is able to replace both (Listing 3). Additionally, if a spine row is missing a timestamp, we can replace the timestamp with a constant $+\infty$ probe column, and the identical join resolves to the newest snapshot by construction.

```
SELECT spine.*, f.total_spend
FROM requests spine
ASOF LEFT JOIN feature_store f
  ON spine.user_id = f.user_id
  AND spine.request_time >= f.__feature_timestamp;
```

Listing 3. A minimal ASOF join underlying `SERVE`.

2.2.2. Cost. DuckDB's ASOF operator is a partitioned sort-merge join: it hash partitions and sorts the right-hand side during the sink phase, hash partitions and sorts the probe rows during the operator phase, and finally matches hash partitions and merge joins the sorted values within each partition. Writing m for the number of spine rows and n for the number of rows on the store side, the two sorts dominate the linear merge pass, giving

$$O(m \log m + n \log n) \ = \ O(\max(m \log m,\ n \log n)).$$

For a store holding E entities at R retained versions each, $n = O(ER)$, so a serve costs

$$O(\max(m \log m,\ ER \log(ER))).$$

3. Implementation

The system of record is the *feature catalog entry*, registered in the schema, and carries the feature's attributes, and information needed to reconstruct the feature. Each feature entry also contains a single dependent denormalized table, `<name>__store`, holding retained snapshots tagged with `__feature_version` and `__feature_timestamp`.

3.1. Feature Creation

`CREATE FEATURE` appends a new feature entry in the catalog. It also acts as a probe query that confirms the timestamp column has a temporal type and that the feature query is bound to capture its result schema while registering every table it touches as a dependency. Entity keys resolve in three tiers: an explicit clause specified in the selection query, then a fully projected `PRIMARY KEY`, then whatever entity columns the query projects.

3.2. Feature Refresh

`REFRESH FEATURE` appends a snapshot and advances the version. It is the only statement with custom logical and physical operators, since it is the only feature statement that writes. It is written as a logical node and a single physical operator that is both sink and source. The sink is parallel, with each thread holding its own optimistic writer and row-group collection. The source then evicts versions outside `RETAIN` and bumps the version *through the catalog*. Routing the bump through the catalog is what makes the append, the eviction and the new version commit atomically, so a crash mid-refresh would roll back the entire operation.

3.3. Feature Serving

`SERVE FEATURE` does an ASOF join with a spine. Unlike `REFRESH FEATURE`, no custom operator is used. The reason is that a native ASOF join with physical and logical operators was already written into DuckDB. As such, to avoid duplicating that logic, the binder assembles the equivalent `SELECT` from the entry and binds it as a subquery.

The optional `TTL` clause bounds how stale a served value is allowed to be: if a spine row's request timestamp is more than `TTL` past the matched snapshot's timestamp, `SERVE` returns NULL for that feature. This is checked at serve time against the matched snapshot's `__feature_timestamp`, so it composes with `RETAIN` without requiring an extra eviction pass: a snapshot can still be physically retained for training purposes while being treated as expired for online serving.

3.4. Persistence and Scheduling

All feature statements round-trip through the Write-Ahead Log and checkpoint, and every alteration produces a new entry, so the feature catalog and stores survive restart. Furthermore, the fork supports a built-in scheduler by exposing an `EVERY` clause.

3.5. Feature Alteration / Dropping

`ALTER FEATURE` is a quality-of-life API over the catalog. It carries no join semantics and no cost model of its own; it simply lets a user modify the attributes of an existing catalog entry — the `TTL`, `EVERY` and `RETAIN` clauses — without dropping and recreating the feature and, with it, the snapshot history already accumulated in the store.

4. Optimizations

Four result-preserving optimizations were added (Table 1), which are gated behind one `feature_disable_optimizations` setting.

Site	Optimized	Baseline
Serve plan choice	Version-table equi-join	Legacy ASOF
Serve scan (legacy)	Bounded by max spine as-of time	Unbounded
GC eviction scan	Zonemap row-group filter	Full scan
Refresh entity spine	`DISTINCT` elided under covering PK	`DISTINCT` kept

TABLE 1. THE FOUR OPTIMIZATION SITES AND THEIR BASELINES.

This report discusses the two optimization areas with the most measured gains: the ASOF Equi-Join and Zonemap-Guided Pruning.

4.1. The ASOF Version-Table Equi-Join

Previously, the `SERVE FEATURE` plan ran an ASOF join between the spine and the entire feature store. As mentioned in §2.2.2, the legacy plan costs $O(m \log m + ER \log(ER))$ — at $E = 17.6$M and $R = 20$, hundreds of millions of rows need to be sorted to answer a lookup.

The key observation is that *a version's timestamp is the same for every entity*: resolving which version a spine row falls under needs only the $(version, timestamp)$ pairs, of which there are at most R. The join is therefore split in two (Listing 4): an ASOF join against a tiny inline list resolves

the version per spine row in $O(m \log m + R \log R)$, and a plain hash equi-join on (entity keys, version) then reaches the store in $O(ER + m)$ expected time. The total becomes

$$O(m \log m + R \log R + ER),$$

which removes the $\log(ER)$ factor over the store.

```
SELECT spine.*, f.* EXCLUDE (...)
FROM <spine> spine
ASOF LEFT JOIN (VALUES (v1,t1), (v2,t2), ...)
     vt(__feature_version, __feature_timestamp)
  ON spine.<ts> >= vt.__feature_timestamp
LEFT JOIN (
   SELECT * FROM <name>__store
   WHERE __feature_version >= <min reachable>
     AND __feature_version <= <max reachable>
) f ON spine.<key> = f.<feature_key>
  AND vt.__feature_version = f.__feature_version;
```

Listing 4. The new version-table equi-join serve plan.

4.2. Zonemap-Guided Pruning

DuckDB's columnar storage stores 122,800 entries in one row-group. Moreover, it keeps per-row-group zonemaps — cheap min/max statistics per column that let a scan skip an entire row group once the zonemap proves no row in it can match a predicate. Because refreshes append snapshots in order, the store table is naturally clustered by `__feature_version` and `__feature_timestamp`, thus making the zonemap predicate filtering even more effective. This shows up in two places:

1) **Serving:** a store row can only be an answer if its timestamp precedes some spine request time, so any row group whose minimum `__feature_timestamp` exceeds the spine's maximum is unreachable and is skipped without being read. Where the reachable range spans $R' \leq R$ versions, the $O(ER)$ term above shrinks to $O(ER')$.
2) **Garbage collection:** deletion in DuckDB is logical until the next checkpoint, so a row evicted by an earlier refresh is still physically present and its row group's zonemap still reports the stale, below-cutoff version. Without a lower bound, that row group is rescanned by every subsequent eviction. Adding `__feature_version >= <oldest live version>`, read from the catalog's version map, prunes those groups out of the search.

5. Benchmark Results

The feature-store benchmark suite exercises DuckDB's `CREATE FEATURE` / `REFRESH FEATURE` / `SERVE FEATURE` surface against the ClickBench *hits* dataset (1/10/100 parquet files (≈1M/10M/100M rows; 79.8K/1.53M/17.6M distinct users)), reshaped as a clickstream feature store keyed by `UserID`. Each test is a point in a knob space sampled by a seeded pairwise covering array, as in randomized DB testing such as SQLancer: the emitted set contains every pair of knob values that can co-occur. More information on the individual test cases can be found here [2].

Three test families are generated from a fixed seed: `serve/` (21 scenarios), `refresh/` (16 scenarios) and `cases/` (6 curated multi-step scenarios). Results are the mean of 3 runs on one machine (Intel Core Ultra 9 185H, 11C/22T, 15 GiB RAM, Ubuntu 22.04 under WSL2).

Family	n	Old (s)	New (s)	Reduction
`serve`	21	168.235	1.164	99.3%
`refresh`	16	9.663	8.427	12.8%
`cases`	18	33.381	25.304	24.2%
Total	55	211.279	34.895	83.5%

TABLE 2. THE ACCUMULATED RUNTIME PER FAMILY, WITH AND WITHOUT OPTIMIZATIONS.

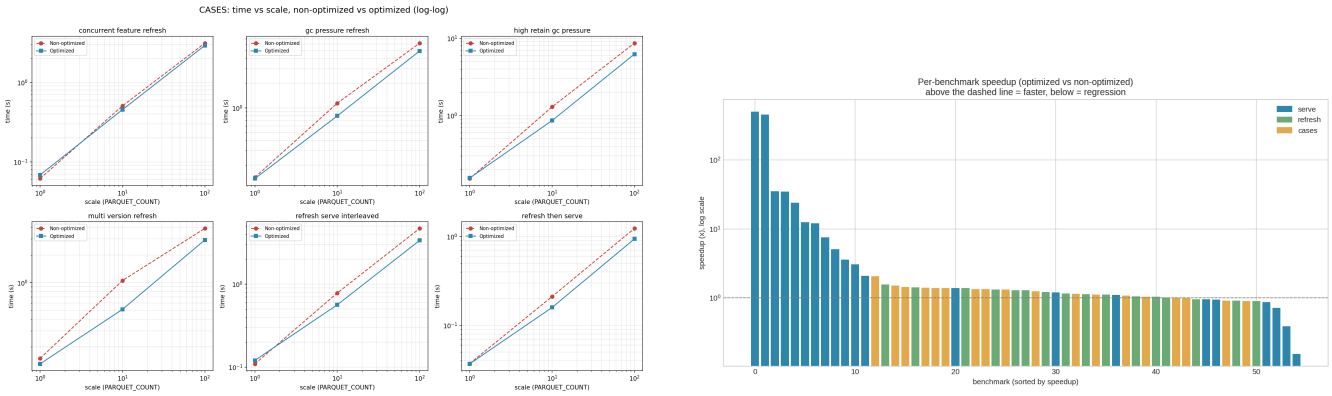
6. Discussion

Serve dominates the total, though the $\log(ER)$ sort factor removed by the ASOF optimization only partly accounts for the margin: at $E = 17.6$M and $R = 20$, it is worth roughly $28\times$, against a measured $504\times$ on the heaviest scenario (133.5 s to 0.265 s; the next falls 26.3 s to 0.057 s). Two further effects compound it. First, the version-range bound and zonemap pruning (§4) mean the equi-join scan touches $O(ER')$ rows for the $R' \leq R$ versions the spine can actually reach, rather than the full store. Second, a hash build avoids the materialization and comparison constants that a sort pays, and a sort over 352M rows against 15 GiB of memory may spill to disk, where the pruned scan does not.

Refresh gains are bounded by construction: the windowed scan and join that dominate a refresh's cost (§2.1.2) are untouched by this optimization, which only prunes the eviction scan for versions outside `RETAIN`. At small scales, there are few evicted row groups to skip, so the bound has nothing to act on; as the source grows, more history accumulates past the retention cutoff and the pruning starts paying for itself, which is why gains only appear once the source is large enough and then cluster at 10–30%, with the curated cases at 24.2% and the GC-pressure scenarios improving 19.7–33.5% at the larger scales.

Of the 55 scenarios, 12 regressed, but each has a baseline under 0.16 s, and ten of the twelve under 0.07 s. Two effects compound at this scale. First, the optimized plans pay fixed setup costs — scalar subqueries for scan bounds, the version table, an extra join — that cannot amortize when the source or spine table is small. Second, sub-0.1 s timings

2. https://github.com/IZO-Ong/duckdb-fs/blob/main/docs/reports/Benchmark_Report.pdf



(a) (b)

Figure 3. The benchmark results: (a) curated cases scaled by source size, simulating realistic end-to-end usage, and (b) per-scenario speedup distribution.

are close enough to scheduling jitter and timer resolution. In aggregate, the regressions cost 0.217 s against 176.4 s saved.

As a final note, the optimizations above were checked directly against the optimizer's actual plan choice: both `REFRESH` and `SERVE` bind to an ordinary logical plan — `REFRESH` to its sink/source operator, `SERVE` to the assembled `SELECT` — so both operations compose with `EXPLAIN`, which was used for this verification.

7. Reflection and Next Steps

This project set out to answer whether a single RDBMS can serve as a self-contained feature store, and the result is a working proof-of-concept: `CREATE`, `REFRESH` and `SERVE FEATURE` run end to end inside DuckDB, correctness is pinned by 26 sqllogictest files, and the optimizations bring the benchmark suite within range of production use.

Three extensions follow directly from what this proof-of-concept demonstrated rather than from what it left unfinished.

1) `REFRESH` and `SERVE` are currently two independent rewrites that happen to share a catalog entry. A possible future step would be to create an operator that can be shared by both — built around the version map — which would remove duplication and improve optimization.
2) The equi-join in §4.1 still resolves `(entity key, version)` through an unindexed hash build over the pruned store scan; an index on the store table keyed by `(<entity key>, __feature_version)` would let the probe side seek directly into the reachable range instead of building a hash table over it on every serve.
3) The temporal-feature model itself need not be restricted to numeric aggregates: a document or image embedding that is recomputed whenever its source changes has the same versioned, point-in-time structure as a windowed `SUM` or `COUNT`, and fits the same `CREATE`/`REFRESH`/`SERVE` surface without new syntax. Whether that generalization holds under embedding-sized payloads and update rates remains an open question.

References

[1] X. Zhou, C. Chen, K. Li, B. He, M. Lu, Q. Liu, W. Wang, X. Cheng, X. Zeng, and Z. Zeng, "FEBench: A benchmark for real-time relational data feature extraction," *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3597–3609, 2023.

[2] J. de la Rúa Martínez, F. Buso, A. Kouzoupis, A. A. Ormenisan, S. Niazi, D. Bzhalava, K. Mak, V. Jouffrey, M. Ronström, R. Cunningham, R. Zangis, D. Mukhedkar, A. Khazanchi, V. Vlassov, and J. Dowling, "The Hopsworks feature store for machine learning," in *Companion of the 2024 International Conference on Management of Data (SIGMOD/PODS '24)*. ACM, 2024, pp. 135–147.